Memorando

De: Jofre Jaime Jamuinda Mutumbungo
Nº de Matrícula: 20230394
Disciplina: Computação Paralela e Distribuída (CPD)
Assunto: Relatório técnico dos testes do Laboratório 2 (OpenMP)
Data: 04/04/2026

---

1. Introdução

Este memorando apresenta os testes realizados no Laboratório 2 da disciplina de CPD, com foco em sincronização de threads e paralelização com OpenMP. Os objectivos principais foram:

1. Analisar o comportamento de execução paralela com diferentes números de threads no problema de sincronização.
2. Observar o impacto de barreiras implícitas e explícitas no fluxo de execução.
3. Identificar dependências de dados e condições de corrida no problema numérico do vector.
4. Implementar versões paralelas correctas com vector auxiliar e com double buffering.

Restrições e observações relevantes:

- O ambiente de compilação utilizado foi gcc, com -fopenmp para habilitar OpenMP.
- A ordem de impressão em programas paralelos não é determinística.
- Nem toda directiva parallel for é válida sem análise de dependências de dados.

2. Experiências Realizadas

2.1 Configuração, build e execução

Comandos de compilação utilizados:

```
gcc omp1.c -fopenmp -o omp1
gcc omp2.c -fopenmp -o omp2
```

Execuções típicas realizadas:

```
export OMP_NUM_THREADS=2  &&  ./omp1
export OMP_NUM_THREADS=3  &&  ./omp1
export OMP_NUM_THREADS=7  &&  ./omp1
export OMP_NUM_THREADS=4  &&  ./omp2
```

2.2 Problema 1 — Sincronização de Threads (omp1.c)

Foi usado o arquivo omp1.c, com região paralela e distribuição de iterações por #pragma omp for. A figura seguinte mostra o código-fonte final com nowait e barrier:

```c
omp1.c > main(int, char * [])
#include<stdio.h>
#include<stdlib.h>
#include<omp.h> // this is a new library

#define NUMITER 30
/* main: process parameters */
int main(int argc, char *argv[]) {

  /* variables */
  int i, tid;

/* create parallel region, make both variable thread private */
#pragma omp parallel private(i,tid)
  {
      /* get id of thread */
      tid = omp_get_thread_num();

/* divide loop iterations evenly by threads */
#pragma omp for nowait
    for(i = 0; i < NUMITER; i++)
      printf("Thread: %d\titer=%d\n", tid, i);  fflush(stdout);
#pragma omp barrier
    /* one of these messages per thread */
    printf("Thread %d, almost..\n", tid);fflush(stdout);
    printf("Thread %d, done!\n", tid);  fflush(stdout);
  }

  /* only master thread prints this one */
  printf("All threads have finished!\n");
}
```

Figura 1 — Código-fonte de omp1.c com #pragma omp for nowait e #pragma omp barrier.

a) Comportamento base — barreira implícita

Observou-se divisão automática das iterações entre threads. Com 2 threads, cada thread recebeu metade das iterações (0–14 e 15–29) de forma sequencial e ordenada. As mensagens “almost” e “done” só aparecem após todas as iterações terminarem, confirmando a barreira implícita do for.

```
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ gcc omp1.c
-fopenmp -o omp1
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ export OMP_
NUM_THREADS=2
./omp1
Thread: 0       iter=0
Thread: 0       iter=1
Thread: 0       iter=2
Thread: 0       iter=3
Thread: 0       iter=4
Thread: 0       iter=5
Thread: 0       iter=6
Thread: 0       iter=7
Thread: 0       iter=8
Thread: 0       iter=9
Thread: 0       iter=10
Thread: 0       iter=11
Thread: 0       iter=12
Thread: 0       iter=13
Thread: 0       iter=14
Thread: 1       iter=15
Thread: 1       iter=16
Thread: 1       iter=17
Thread: 1       iter=18
Thread: 1       iter=19
Thread: 1       iter=20
Thread: 1       iter=21
Thread: 1       iter=22
Thread: 1       iter=23
Thread: 1       iter=24
Thread: 1       iter=25
Thread: 1       iter=26
Thread: 1       iter=27
Thread: 1       iter=28
Thread: 1       iter=29
Thread 0, almost..
Thread 0, done!
Thread 1, almost..
Thread 1, done!
```

Figura 2 — Compilação e execução de omp1 com OMP_NUM_THREADS=2 (barreira implícita, alínea a).

b) Uso de nowait — remoção da barreira implícita

Ao remover a barreira implícita do for com a cláusula nowait, algumas threads imprimiram “almost” e “done” antes de outras terminarem as suas iterações. A saída tornou-se não determinística e o interleaving das mensagens ficou visível com 3 threads.

```
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ export OMP_
NUM_THREADS=3
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ ./omp1
Thread: 0       iter=0
Thread: 0       iter=1
Thread: 0       iter=2
Thread: 0       iter=3
Thread: 0       iter=4
Thread: 2       iter=20
Thread: 2       iter=21
Thread: 2       iter=22
Thread: 2       iter=23
Thread: 2       iter=24
Thread: 2       iter=25
Thread: 2       iter=26
Thread: 2       iter=27
Thread: 2       iter=28
Thread: 0       iter=5
Thread: 0       iter=6
Thread: 0       iter=7
Thread: 0       iter=8
Thread: 0       iter=9
Thread: 1       iter=10
Thread: 1       iter=11
Thread: 1       iter=12
Thread: 1       iter=13
Thread: 1       iter=14
Thread: 1       iter=15
Thread: 1       iter=16
Thread: 1       iter=17
Thread: 1       iter=18
Thread: 1       iter=19
Thread: 2       iter=29
Thread 1, almost..
Thread 1, done!
Thread 2, almost..
Thread 2, done!
Thread 0, almost..
Thread 0, done!
```

Figura 3 — Execução de omp1 com OMP_NUM_THREADS=3 e nowait — interleaving das mensagens finais (alínea b).

c) Uso de #pragma omp barrier — sincronização explícita

Com a directiva #pragma omp barrier inserida entre o ciclo for e as mensagens finais, a sincronização foi restabelecida: todas as threads completam o ciclo antes de avançar. A tabela seguinte resume a comparação entre os três casos:

Caso	Comportamento
a) Barreira implícita	Sincronização automática no final do for
b) nowait	Sem sincronização — threads avançam livremente
c) barrier explícita	Sincronização manual — comportamento equivalente a a)

Com 7 threads, o alto grau de paralelismo produziu saída com forte interleaving das iterações, conforme ilustrado abaixo:

```
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ export OMP_
NUM_THREADS=7
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ ./omp1
Thread: 3       iter=14
Thread: 3       iter=15
Thread: 3       iter=16
Thread: 3       iter=17
Thread: 2       iter=10
Thread: 2       iter=11
Thread: 2       iter=12
Thread: 2       iter=13
Thread: 6       iter=26
Thread: 6       iter=27
Thread: 6       iter=28
Thread: 6       iter=29
Thread: 1       iter=5
Thread: 1       iter=6
Thread: 1       iter=7
Thread: 1       iter=8
Thread: 1       iter=9
Thread: 0       iter=0
Thread: 0       iter=1
Thread: 0       iter=2
Thread: 0       iter=3
Thread: 0       iter=4
Thread: 4       iter=18
Thread: 4       iter=19
Thread: 4       iter=20
Thread: 4       iter=21
Thread: 5       iter=22
Thread: 5       iter=23
Thread: 5       iter=24
Thread: 5       iter=25
Thread 5, almost..
Thread 1, almost..
Thread 3, almost..
Thread 3, done!
Thread 5, done!
```

Figura 4 — Execução de omp1 com OMP_NUM_THREADS=7 — alto interleaving (parte 1).

```
Thread: 2       iter=13
Thread: 6       iter=26
Thread: 6       iter=27
Thread: 6       iter=28
Thread: 6       iter=29
Thread: 1       iter=5
Thread: 1       iter=6
Thread: 1       iter=7
Thread: 1       iter=8
Thread: 1       iter=9
Thread: 0       iter=0
Thread: 0       iter=1
Thread: 0       iter=2
Thread: 0       iter=3
Thread: 0       iter=4
Thread: 4       iter=18
Thread: 4       iter=19
Thread: 4       iter=20
Thread: 4       iter=21
Thread: 5       iter=22
Thread: 5       iter=23
Thread: 5       iter=24
Thread: 5       iter=25
Thread 5, almost..
Thread 1, almost..
Thread 3, almost..
Thread 3, done!
Thread 5, done!
Thread 1, done!
Thread 6, almost..
Thread 6, done!
Thread 2, almost..
Thread 2, done!
Thread 4, almost..
Thread 4, done!
Thread 0, almost..
Thread 0, done!
All threads have finished!
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$
```

Figura 5 — Execução de omp1 com OMP_NUM_THREADS=7 — mensagens finais ordenadas após barrier (parte 2).

```
NUM_THREADS=2
./omp1
Thread: 0       iter=0
Thread: 0       iter=1
Thread: 0       iter=2
Thread: 0       iter=3
Thread: 0       iter=4
Thread: 0       iter=5
Thread: 0       iter=6
Thread: 0       iter=7
Thread: 0       iter=8
Thread: 0       iter=9
Thread: 0       iter=10
Thread: 0       iter=11
Thread: 0       iter=12
Thread: 0       iter=13
Thread: 0       iter=14
Thread: 1       iter=15
Thread: 1       iter=16
Thread: 1       iter=17
Thread: 1       iter=18
Thread: 1       iter=19
Thread: 1       iter=20
Thread: 1       iter=21
Thread: 1       iter=22
Thread: 1       iter=23
Thread: 1       iter=24
Thread: 1       iter=25
Thread: 1       iter=26
Thread: 1       iter=27
Thread: 1       iter=28
Thread: 1       iter=29
Thread 0, almost..
Thread 0, done!
Thread 1, almost..
Thread 1, done!
All threads have finished!
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$
```

Figura 6 — Execução de omp1 com OMP_NUM_THREADS=2 e barrier explícita — sincronização correcta (alínea c).

2.3 Problema 2 — Dependência de Dados (omp2.c)

Foi analisado o algoritmo de actualização iterativa de um vector com a função:

```
#define f(x,y) ((x+y)/2.0)
V[i] = f(V[i], V[i+1]);
```

a) Versão serial

A versão serial do código foi compilada e executada sem qualquer directiva OpenMP. O resultado foi determinístico e correcto — os valores do vector convergiram após as iterações, com propagação da influência dos elementos da direita para a esquerda.

```c
omp2.c > main(int, char * [])
/* MAIN: PROCESS PARAMETERS */
int main(int argc, char *argv[]) {

  /* VARIABLES */
  int i, iter;

  /* DECLARE VECTOR AND AUX DATA STRUCTURES */
  double *V = (double *) malloc(TOTALSIZE * sizeof(double));

  /* 1. INITIALIZE VECTOR */
  for(i = 0; i < TOTALSIZE; i++) {
    V[i]= 0.0 + i;
  }

  /* 2. ITERATIONS LOOP */
  for(iter = 0; iter < NUMITER; iter++) {

    /* 2.1. PROCESS ELEMENTS */
    for(i = 0; i < TOTALSIZE-1; i++) {
      V[i] = f(V[i], V[i+1]);
    }

    /* 2.2. END ITERATIONS LOOP */
  }

  /* 3. OUTPUT FINAL VALUES */
  printf("Output:\n");
  for(i = 0; i < TOTALSIZE; i++) {
    printf("%4d %f\n", i, V[i]);
  }

}
```

Figura 7 — Código-fonte de omp2.c versão serial.

```
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ ./omp2
-fopenmp -o omp2
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/cpd-lab2$ ./omp2
```

Figura 8 — Compilação e execução da versão serial de omp2.

```
 78 178.000000
 79 179.000000
 80 180.000000
 81 181.000000
 82 182.000000
 83 183.000000
 84 184.000000
 85 185.000000
 86 186.000000
 87 187.000000
 88 188.000000
 89 189.000000
 90 190.000000
 91 191.000000
 92 192.000000
 93 193.000000
 94 194.000000
 95 195.000000
 96 196.000000
 97 197.000000
 98 198.000000
 99 199.000000
100 200.000000
101 201.000000
102 202.000000
103 203.000000
104 204.000000
105 205.000000
106 206.000000
107 207.000000
108 208.000000
109 209.000000
110 210.000000
111 211.000000
112 212.000000
113 213.000000
114 214.000000
115 215.000000
116 216.000000
```

Figura 9 — Saída correcta da versão serial de omp2 (valores determinísticos).

b) Paralelização incorrecta directa

Ao aplicar #pragma omp parallel for directamente no ciclo interno, surgiram resultados diferentes da versão serial e não determinísticos. A causa é uma condição de corrida: múltiplas threads leram e escreveram V[i] e V[i+1] concorrentemente, sem garantia da ordem de acesso.

```c
omp2.c > main(int, char * [])
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>   // 🔥 adicionar isto

#define TOTALSIZE 1000
#define NUMITER 200

#define f(x,y) ((x+y)/2.0)

int main(int argc, char *argv[]) {

  int i, iter;

  double *V = (double *) malloc(TOTALSIZE * sizeof(double));

  for(i = 0; i < TOTALSIZE; i++) {
    V[i]= i;
  }

  for(iter = 0; iter < NUMITER; iter++) {

    #pragma omp parallel for   // 🔥 AQUI está a mudança
    for(i = 0; i < TOTALSIZE-1; i++) {
      V[i] = f(V[i], V[i+1]);
    }

  }

  printf("Output:\n");
  for(i = 0; i < TOTALSIZE; i++) {
    printf("%4d %f\n", i, V[i]);
  }
}
```

Figura 10 — Código-fonte da versão paralela incorrecta de omp2 com #pragma omp parallel for.

```
Output:
   0 100.000125
   1 101.000211
   2 102.000350
   3 103.000570
   4 104.000909
   5 105.001421
   6 106.002181
   7 107.003283
   8 108.004849
   9 109.007031
  10 110.010009
  11 111.013991
  12 112.019209
  13 113.025910
  14 114.034342
  15 115.044741
  16 116.057312
  17 117.072205
  18 118.089501
  19 119.109188
  20 120.131156
  21 121.155182
  22 122.180943
  23 123.208018
  24 124.235913
  25 125.264087
```

Figura 11 — Saída incorrecta e não determinística da versão paralela directa (race condition).

c) Correção com vector auxiliar

A correção foi implementada copiando o estado anterior para V_old antes de cada iteração, e lendo apenas de V_old no cálculo paralelo. Isto elimina a dependência de dados, pois não há escrita concorrente sobre dados utilizados para leitura.

```
for(i = 0; i < TOTALSIZE; i++) { V_old[i] = V[i]; }
#pragma omp parallel for
for(i = 0; i < TOTALSIZE-1; i++) { V[i] = f(V_old[i], V_old[i+1]); }
```

d) Optimização com double buffering

Para evitar a cópia completa do vector a cada iteração, foi implementada a técnica de double buffering com dois vectores (V1 e V2) e troca de ponteiros. A troca é O(1) e elimina a necessidade de cópia:

```
double *temp = read;
read = write;
write = temp;
```

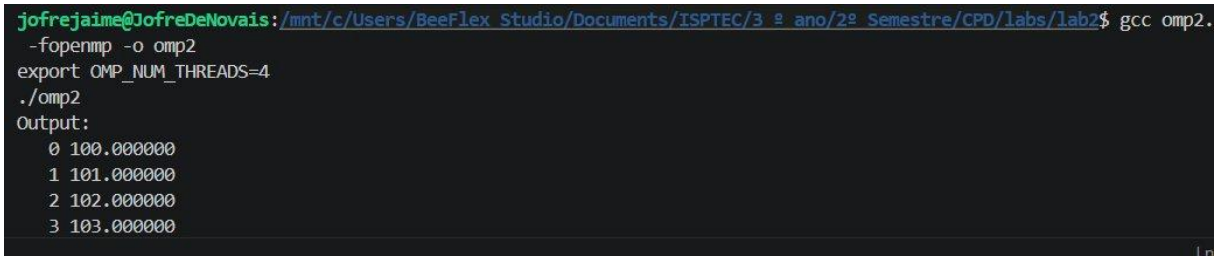

Figura 12 — Execução correcta da versão double buffering de omp2 com OMP_NUM_THREADS=4.

3. Desafios

Panoâmica do software concebido e implementado:

5. Estrutura em dois programas independentes: omp1.c (sincronização) e omp2.c (dependências de dados).
6. Evolução incremental das soluções: serial → paralelo incorrecto → paralelo correcto com cópia → paralelo correcto optimizado com double buffering.
7. Principal desafio técnico: identificar que a dependência V[i] ← V[i+1] inviabiliza a paralelização directa do ciclo interno.
8. Principal decisão de engenharia: separar os dados de leitura e escrita por iteração para garantir a correcção do algoritmo paralelo.

4. Referências Bibliográficas

9. OpenMP Architecture Review Board. OpenMP API Specification, versão 4.5. Disponível em: http://www.openmp.org/wp-content/uploads/openmp-4.5.pdf
10. GCC Documentation. Options for OpenMP (-fopenmp). Disponível em: https://gcc.gnu.org/onlinedocs/
11. Material de apoio da disciplina CPD — ISPTEC, Coordenação de Engenharia Informática.
12. Código base e enunciado do Laboratório 2 (ficheiros omp1.c e omp2.c fornecidos pelo docente).

5. Repositório GitHub

Link do repositório: https://github.com/Jofrejaime/computacao-paralela-e-destribuida

Observação: convite de colaborador enviado para o utilizador GitHub joaojdacosta.